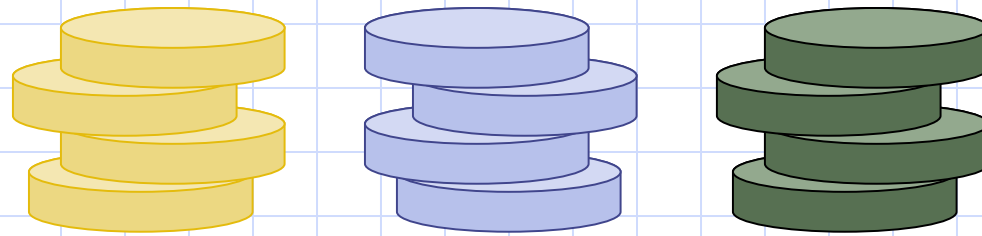


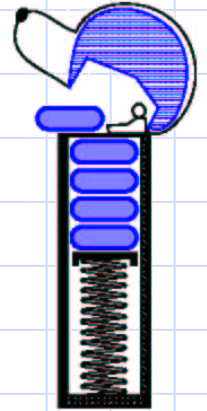
# Stacks



# Abstract Data Types (ADTs)

- An abstract data type (ADT) is an abstraction of a data structure
- An ADT specifies:
  - Data stored
  - Operations on the data
  - Error conditions associated with operations
- Example: ADT modeling a simple stock trading system
  - The data stored are buy/sell orders
  - The operations supported are
    - ♦ order **buy**(stock, shares, price)
    - ♦ order **sell**(stock, shares, price)
    - ♦ void **cancel**(order)
  - Error conditions:

# The Stack ADT



- The **Stack** ADT stores arbitrary objects
- Insertions and deletions follow the last-in first-out scheme
- Think of a spring-loaded plate dispenser
- Main stack operations:
  - **push**(object): inserts an element
  - object **pop**(): removes and returns the last inserted element
- Auxiliary stack operations:
  - object **top**(): returns the last inserted element without removing it
  - integer **len**(): returns the number of elements stored
  - boolean **is\_empty**(): indicates whether no elements are stored

# Example

| Operation    | Return Value | Stack Contents |
|--------------|--------------|----------------|
| S.push(5)    | —            | [5]            |
| S.push(3)    | —            | [5, 3]         |
| len(S)       | 2            | [5, 3]         |
| S.pop()      | 3            | [5]            |
| S.is_empty() | False        | [5]            |
| S.pop()      | 5            | []             |
| S.is_empty() | True         | []             |
| S.pop()      | “error”      | []             |
| S.push(7)    | —            | [7]            |
| S.push(9)    | —            | [7, 9]         |
| S.top()      | 9            | [7, 9]         |
| S.push(4)    | —            | [7, 9, 4]      |
| len(S)       | 3            | [7, 9, 4]      |
| S.pop()      | 4            | [7, 9]         |
| S.push(6)    | —            | [7, 9, 6]      |
| S.push(8)    | —            | [7, 9, 6, 8]   |
| S.pop()      | 8            | [7, 9, 6]      |

# Applications of Stacks

- ❑ Direct applications
  - Page-visited history in a Web browser
  - Undo sequence in a text editor
  - Chain of method calls in a language that supports recursion
- ❑ Indirect applications
  - Auxiliary data structure for algorithms
  - Component of other data structures

# Array-based Stack

- A simple way of implementing the Stack ADT uses an array
- We add elements from left to right
- A variable keeps track of the index of the top element



# Array-based Stack (cont.)

- The array storing the stack elements may become full
- A push operation will then need to grow the array and copy all the elements over.



# Performance and Limitations

## □ Performance

- Let  $n$  be the number of elements in the stack
- The space used is  $O(n)$
- Each operation runs in time  $O(1)$  (amortized in the case of a push)



# Array-based Stack in Python



```
1 class ArrayStack:
2     """LIFO Stack implementation using a Python list as underlying storage."""
3
4     def __init__(self):
5         """Create an empty stack."""
6         self._data = [ ]           # nonpublic list instance
7
8     def __len__(self):
9         """Return the number of elements in the stack."""
10        return len(self._data)
11
12    def is_empty(self):
13        """Return True if the stack is empty."""
14        return len(self._data) == 0
15
16    def push(self, e):
17        """Add element e to the top of the stack."""
18        self._data.append(e)        # new item stored at end of list
19
```

```
20    def top(self):
21        """Return (but do not remove) the element at the top of the stack.
22
23        Raise Empty exception if the stack is empty.
24        """
25        if self.is_empty():
26            raise Empty('Stack is empty')
27        return self._data[-1]       # the last item in the list
28
29    def pop(self):
30        """Remove and return the element from the top of the stack (i.e., LIFO).
31
32        Raise Empty exception if the stack is empty.
33        """
34        if self.is_empty():
35            raise Empty('Stack is empty')
36        return self._data.pop( )    # remove last item from list
```

# Parentheses Matching

- Each “(”, “{”, or “[” must be paired with a matching “)”, “}”, or “]”
  - correct: `()(( )){([ ( ))}`
  - Correct: `(( ))( )(( )){([ ( ))}`
  - incorrect: `)(( )){([ ( ))}`
  - incorrect: `{([ ])}`
  - incorrect: `(`

# Parentheses Matching Algorithm

**Algorithm** ParenMatch( $X, n$ ):

**Input:** An array  $X$  of  $n$  tokens, each of which is either a grouping symbol, a variable, an arithmetic operator, or a number

**Output:** **true** if and only if all the grouping symbols in  $X$  match

Let  $S$  be an empty stack

**for**  $i=0$  to  $n-1$  **do**

**if**  $X[i]$  is an opening grouping symbol **then**

$S.push(X[i])$

**else if**  $X[i]$  is a closing grouping symbol **then**

**if**  $S.is\_empty()$  **then**

**return false** {nothing to match with}

**if**  $S.pop()$  does not match the type of  $X[i]$  **then**

**return false** {wrong type}

**if**  $S.isEmpty()$  **then**

**return true** {every symbol matched}

**else return false** {some symbols were never matched}

# Parentheses Matching in Python

```
1 def is_matched(expr):
2     """Return True if all delimiters are properly match; False otherwise."""
3     lefty = '([{' # opening delimiters
4     righty = ')]}' # respective closing delims
5     S = ArrayStack()
6     for c in expr:
7         if c in lefty:
8             S.push(c) # push left delimiter on stack
9         elif c in righty:
10            if S.is_empty():
11                return False # nothing to match with
12            if righty.index(c) != lefty.index(S.pop()):
13                return False # mismatched
14    return S.is_empty() # were all symbols matched?
```

# HTML Tag Matching

- ◆ For fully-correct HTML, each `<name>` should pair with a matching `</name>`

`<body>`

`<center>`

`<h1> The Little Boat </h1>`

`</center>`

`<p>` The storm tossed the little boat like a cheap sneaker in an old washing machine. The three drunken fishermen were used to such treatment, of course, but not the tree salesman, who even as a stowaway now felt that he had overpaid for the voyage. `</p>`

`<ol>`

`<li>` Will the salesman die? `</li>`

`<li>` What color is the boat? `</li>`

`<li>` And what about Naomi? `</li>`

`</ol>`

`</body>`

## The Little Boat

The storm tossed the little boat like a cheap sneaker in an old washing machine. The three drunken fishermen were used to such treatment, of course, but not the tree salesman, who even as a stowaway now felt that he had overpaid for the voyage.

1. Will the salesman die?
2. What color is the boat?
3. And what about Naomi?

# Tag Matching Algorithm in Python

```
1 def is_matched_html(raw):
2     """Return True if all HTML tags are properly match; False otherwise."""
3     S = ArrayStack()
4     j = raw.find('<')           # find first '<' character (if any)
5     while j != -1:
6         k = raw.find('>', j+1)   # find next '>' character
7         if k == -1:
8             return False        # invalid tag
9         tag = raw[j+1:k]        # strip away < >
10        if not tag.startswith('/'): # this is opening tag
11            S.push(tag)
12        else:                    # this is closing tag
13            if S.is_empty():
14                return False      # nothing to match with
15            if tag[1:] != S.pop():
16                return False      # mismatched delimiter
17            j = raw.find('<', k+1)  # find next '<' character (if any)
18    return S.is_empty()         # were all opening tags matched?
```